VTX1 Ternary SoC Architecture

K.Vanyushov

Version 2.0, 2026-10-18: Trit-native rework - spec rewritten from the implemented machine

Table of Contents

Authors . . . 1
Overview . . . 2
- What VTX1 is . . . 2
- Design principles (as practiced) . . . 2
- The number system . . . 2
- Physical trit encoding . . . 2
- Operation words and the opcode space . . . 3
- Formats . . . 3
- Registers . . . 3
- Ratified semantics . . . 4
- VLIW bundles . . . 4
- Trapping operations . . . 4
- Issue slots . . . 4
- Dynamic issue rules . . . 4
- Fetch . . . 5
- Execution units . . . 5
- State machine . . . 5
- The matched binary control . . . 5
- Tightly-coupled instruction memory (TCM) . . . 6
- Data scratchpad . . . 6
- Bus and main memory . . . 6
- Cache . . . 6
- Memory-mapped I/O at negative addresses . . . 6
- The ternary-weight MAC accelerator . . . 7
- Verification harnesses . . . 7
- Toolchain . . . 7
- The evaluation method . . . 8
- The honest verdict . . . 8
- Known gaps . . . 8

Authors


```
K. Vanyushov (itworks99)
Lead Architect and Maintainer of the VTX1 Project
GitHub: https://github.com/itworks99
Email: kvanyushov@gmail.com
```


NOTE

This revision describes the machine as implemented and measured, not as planned. Every structure named here exists in the repository's RTL, executes the same binaries as the ISA simulator, and synthesizes with Yosys; every performance number is produced by a committed task. The pre-2026 microcode-era chapters are archived under `docs/archive/`. The running design-decision log is `src/assembler/todo.md`.

Overview

What VTX1 is

VTX1 is a balanced-ternary System-on-Chip: binary on the outside, ternary on the inside. Buses, memories, files and I/O carry trits in a 2-bit hardware encoding; the instruction set, registers, arithmetic and addressing are native balanced ternary (trits −1, 0, +1; 18-trit machine words). The project's thesis is that a three-valued, symmetric number system buys measurable architectural properties - and the project's method is to measure them honestly, with matched controls, at every level from instruction counts down to SPICE.

Design principles (as practiced)

- **Trit-native, not trit-themed.** The ISA, immediates, branch offsets and the entire datapath are balanced ternary. The only trit-to-binary conversion in the CPU is at the memory address pins. Binary instruction encodings were rejected as a deficiency and removed.
- **One source of truth.** The complete ISA lives in `src/assembler/isa/isa.go`. The Verilog opcode constants and decode tables are generated from it, so the assembler, simulator and RTL cannot drift apart.
- **No microcode.** Operations are single-cycle hardwired logic or dedicated multi-cycle units (the iterative divider). The microcode layer of the original design was retired.
- **Cross-checked at every seam.** The same binary runs in the ISA simulator and on the RTL, and retired-instruction counts must match exactly. Benchmark results are asserted by tests on both sides of every comparison.
- **Evaluation by simulation tiers, not FPGA.** An FPGA cannot be ternary, so the architecture is evaluated where the substrate cannot distort it: instruction-level (Tier A), cycle-level RTL against matched binary controls, gate-level synthesis, and SPICE on production Sky130 device models (Tier B).

The number system

Balanced ternary with 18-trit words gives a symmetric value range of $\pm(3^{18}-1)/2 = \pm 193{,}710{,}244$ and these properties, all exploited by the hardware:

- Negation is tritwise (swap −1 and +1): subtraction is addition, and NOT and NEG are the same operation.
- There is no sign bit and no two's-complement asymmetry; "unsigned" does not exist.
- Zero-trit padding preserves any value - no sign extension exists.
- Truncation rounds to nearest; the iterative divider naturally produces round-to-nearest quotients.
- Dropped adder carries implement arithmetic mod 3^{18}; a nonzero final carry trit is the true sign of an overflowed difference, which gives overflow-correct three-way compare for free.
- Multiplier digits are already in {−1, 0, +1}: multiplication is natively Booth-recoded.

Physical trit encoding

Each trit is stored as 2 bits: `00` = −1, `01` = 0, `10` = +1, `11` = undefined (an instruction containing it traps). An 18-trit word occupies 36 bits; files store words as 5 bytes. Tier B (SPICE on Sky130) showed this encoding is not a stopgap: at the gate level it is cheaper than voltage-mode ternary CMOS, so the binary-encoded

trit datapath is the deliberate engineering choice (see the Evaluation chapter). == Instruction Set Architecture

The normative definition is `src/assembler/isa/isa.go`; this chapter describes it. The Verilog headers (`vtx1_isa_generated.vh`, `vtx1_isa_decode.vh`) are generated from that file with `go run ./isa/gen`.

Operation words and the opcode space

Every operation is one 18-trit word. Trit 0 is the most significant and occupies bits [35:34]. All formats start with a 4-trit opcode (trits 0-3), giving an 81-value space: 69 assigned, 12 reserved (the retired transcendental and unsigned operations). The opcode's numeric value is its balanced value + 40. The all-zero-trit word is NOP, so zeroed memory executes as NOPs.

Formats

Format	Layout (trit positions, MSB first)
R3	rd 4-6 | rs1 7-9 | mode 10 | payload 11-17 mode 0: rs2 = trits 15-17; mode +1: 7-trit immediate (±1093)
R2	rd 4-6 | rs 7-9
R1	r 4-6
None	all operand trits zero
Mem	reg 4-6 | mode 7 | payload 8-17 mode 0: 10-trit direct address/immediate (±29524) mode +1: base 8-10, index 11-13 (`[base]` uses index TZ) mode −1: base 8-10, 7-trit offset 11-17
Branch	rs1 4-6 | rs2 7-9 | 8-trit PC-relative word offset 10-17 (±3280); rs2 = TZ encodes compare-against-zero
Jump	14-trit absolute word address, trits 4-17 (±2391484)

All numeric fields are balanced ternary: immediates, offsets and displacements are naturally signed and symmetric around zero.

Registers

A unified 3-trit index space (pattern = balanced(index − 13)):

Registers	Indices	Role
T0-T6	0-6	general purpose
TB	7	stack/base pointer (the ratified stack ABI)
TA	8	accumulator and link register (CALL/JAL/JALR write the return address here; RET jumps to it)
TC	9	program counter
TS	10	status
TI	11	instruction
VA, VT, VB	12-14	vector (operations currently trap)
FA, FT, FB	15-17	floating point (operations currently trap)
TZ	18	zero register: reads 0, writes discarded

Ratified semantics

These decisions are binding on every implementation (simulator and RTL enforce them identically):

- **LD is a memory read; LEA is load-immediate/address.** The effective address of a direct operand is the immediate.
- **Division truncates toward zero;** MOD's sign follows the dividend; `a/b*b + a%b == a`; division by zero traps.
- **Shift/rotate amounts** are the value of the low 4 trits of the second source (±40), reduced mod 18; negative amounts shift the opposite way.
- **Stack ABI:** TB is the stack pointer, full-descending, word-addressed. `PUSH r`: TB ← TB−1, mem[TB] ← r (the old r, relevant for PUSH TB). `POP r`: r ← mem[TB], TB ← TB+1. For `POP TB` the loaded value wins. Programs initialize TB themselves.
- **Negative addresses are memory-mapped I/O** (see the SoC chapter for the ratified peripheral map). Positive addresses are word-addressed memory: one address names one 18-trit word.
- Arithmetic wraps mod 3^{18} (dropped carries). Compare (CMP and the branch comparisons) is overflow-correct: a nonzero carry-out trit of the subtraction is taken as the true sign.

VLIW bundles

A bundle is 3 operation words = 54 trits = 108 bits. The assembler enforces, per bundle: at most 3 operations (shorter bundles are NOP-padded), at most one memory operation, at most one control operation, no two writes to the same register (error), and a warning when a slot reads a register another slot writes. The core enforces the same rules dynamically at issue time (see the CPU chapter), so a packed bundle is a scheduling guarantee, not a correctness obligation: scalar code multi-issues whenever adjacent words happen to be independent.

Trapping operations

SQRT, PUSH-era vector/FP operations (VLD/VST/FLD/FST and the vector/FP ALU set) trap as unimplemented. An instruction word containing an undefined trit pair traps. Traps halt the core with `trap_pc` pointing at the offending word. == CPU Microarchitecture

The core (`src/cpu/vtx1_core.v`) is a triple-issue, conversion-free balanced-ternary machine: registers, PC, immediates, effective addresses and all results are trit words end to end. The only trit-to-binary conversion is `t2b()` toward the memory address bus (a chain of ×3 adds); there is no binary-to-trit conversion anywhere in the datapath.

Issue slots

The core instantiates three `vtx1_slot` units. Each slot decodes and combinationally executes one operation word against the architectural register file: its ALU result, effective address, branch decision, control-flow target, plus the classification and hazard metadata (destination and source register indices) the issue logic consumes. A slot never writes state; the core commits only the slots it issues.

Dynamic issue rules

Up to three **consecutive** words commit per cycle. The hardware enforces the VLIW packing rules itself:

- at most one memory operation per group;
- a control operation (branch/jump/RET/JR/JALR) or halt ends the group;
- DIV/MOD and PUSH/POP issue alone (multi-cycle, or two destinations);
- no intra-group RAW or WAW (TZ never conflicts): dependent words simply start the next group.

Co-issued words are therefore provably independent, and parallel commit is bit-identical to sequential execution - which is why the ISA simulator's retired-word count cross-check holds exactly.

Fetch

Instruction fetch has a fast path through the banked tightly-coupled instruction memory (see the Memory chapter): during EXEC the next PC is resolved combinationally - branch targets included - and the TCM read for the next 3-word group is issued in the same cycle. TCM-resident code therefore needs no fetch state at all: straight-line independent code runs at up to 3 words per cycle, and taken branches cost zero extra cycles. After a memory operation or divide, the already-fetched group is still held in the TCM output registers, so the core resumes execution with no refetch cycle. A TCM miss falls back to the bus port, one word per fetch, single-issue.

Execution units

- **ALU** (`vtx1_alu.v`, one per slot): built from a balanced-ternary full-adder cell (`vtx1_trit_fa`) and an 18-trit ripple adder with tritwise negation on one input. ADD/SUB/MUL (an 18-partial product tree - Booth recoding is free in balanced ternary), AND = min, OR = max, XOR = product, NOT = NEG = tritwise negation, ABS, CMP and TEST, shifts and rotates, INC/DEC. Compare uses the overflow-correct rule: a nonzero carry-out trit of the subtraction is the true sign.
- **Iterative divider** (`vtx1_div.v`, shared): DIV/MOD in ~22 cycles. It operates on magnitudes (signs are applied for free by tritwise negation at the end), retiring one quotient digit per trit MSB-first with digits in {−2..2} on a 20-trit internal datapath; the natural result is the round-to-nearest quotient, converted to truncation by one correction step. Division by zero is trapped by the core before start.

State machine

FETCH, EXEC, LOAD, STORE, DIV and HALT. EXEC chains into EXEC on the TCM fast path. LOAD/STORE hold a one-port word-addressed memory request with a ready handshake, so the core connects equally to a simple synchronous RAM or the SoC bus matrix; store data is latched at issue. The core exposes `retired` (instruction words, cross-checked against the simulator) and `cycles` (stops counting at halt).

The matched binary control

`src/control/rv32_triple/` is this exact microarchitecture with the trit datapath replaced by two's-complement RV32IM: the same slots, issue rules, FSM, TCM structure and memory model, by the same author. A cycle difference between the two cores isolates the ISA; the Evaluation chapter gives the results. == Memory System

Memory is word-addressed: one address names one 18-trit word (36 bits in the 2-bit encoding; 5 bytes in files). A scalar instruction advances the PC by 1, a VLIW bundle by 3. Positive addresses are memory; negative addresses are memory-mapped I/O.

Tightly-coupled instruction memory (TCM)

src/memory/vtx1_tcm.v serves the core's dedicated fetch port with a single-cycle synchronous read of **three consecutive words** (the issue group), covering the program region 0x1000-0x1FFF (4096 words). The array is interleaved across four banks on the low address bits, so any three consecutive addresses - arbitrary alignment, branch targets included - read in one cycle. Its write port snoops the CPU's store path (write-through), which keeps CPU-originated writes coherent and doubles as the boot-time load path: the boot flow loads code with ordinary stores. Writes by other bus masters are not snooped.

Data scratchpad

src/memory/vtx1_scratchpad.v sits between the core's memory port and the SoC bus master port, covering the low 16K words. In-window **reads** are served from a local synchronous RAM with a one-cycle handshake instead of the multi-cycle bus matrix round trip. **Writes** always pass through to the bus - external memory remains the source of truth for DMA, debug and verification - and are snooped into the local copy. Out-of-window accesses (including all MMIO) pass through untouched.

The scratchpad also exposes a second read port and a shared write port to the MAC accelerator's DMA engine (CPU snoop writes have priority). Ownership caveat, stated: words the accelerator writes exist only in the scratchpad until a CPU store overwrites that address.

Bus and main memory

The core's bus port is master 0 on the bus matrix (src/system/bus_matrix.v), alongside the DMA and debug controllers. The matrix routes the MMIO window to the peripheral router and everything else to the memory controller (src/memory/memory_controller.v), which fronts the cache controller and the external SRAM pins. The SoC testbench models a 16K-word external SRAM on those pins; hello_world and all benchmark programs execute through this full path.

Cache

src/cache/cache_controller.v implements direct-mapped instruction and data caches (64 entries each). The data and tag arrays are written in a reset-free synchronous block so synthesis infers RAMs; only the valid and dirty bits are registers. With the TCM and scratchpad carrying the hot paths, the cache is not performance-critical in the current memory map. == System-on-Chip

src/vtx1_soc.v integrates the core with its TCM and data scratchpad, the bus matrix (CPU, DMA controller and debug controller as masters), the memory controller, cache, external SRAM pins, and the MMIO peripherals: GPIO, UART, SPI, I2C, flash controller, the JTAG debug controller, clock/reset/power management - and the ternary-weight MAC accelerator. The whole SoC synthesizes with Yosys (task synth-soc); the smoke test shrinks the TCM and scratchpad arrays with chparam, since the generic flow maps RAMs to flip-flops.

Memory-mapped I/O at negative addresses

The ISA reserves negative word addresses for MMIO, and the SoC implements the mapping: for a negative address of magnitude m, peripheral = m[19:16] and register offset = m[15:0] - that is, peripheral P register k lives at word address –(P·0x10000 + k).

P	Peripheral	Notes
0	GPIO	24-pin controller

P	Peripheral	Notes
1	UART	with DMA request lines
2	SPI	
3	I2C	
4	TIMER	address decode reserved
5	FLASH	SPI flash controller
6	TMAC	ternary-weight MAC accelerator

The ternary-weight MAC accelerator

`src/accel/vtx1_tmac.v` is 16 multiply-accumulate lanes with no multipliers: weights are trits, so a weight multiply is tritwise select / skip / negate (negation is a wire swap), feeding a tree of the ALU's 18-trit adders. Arithmetic wraps mod 3^{18} exactly like the CPU's ADD chain, so software and accelerator results are bit-identical. One machine word carries all 16 lane weights in its low 16 trits (lane i at weight 3^i): a weight row is the balanced number $\Sigma\, w_i \cdot 3^i$, emitted by an ordinary `.DW`.

The MMIO wrapper (`vtx1_tmac_mmio.v`, peripheral 6) offers two modes:

- **Direct:** activation words at offsets 0x00-0x0F, accumulator read/clear at 0x10, a weight-word write to 0x11 = one 16-lane step.
- **DMA:** descriptor registers ACT_BASE / W_BASE / R_BASE at 0x12-0x14, row count + START at 0x15 (which also reads back as the busy status). The engine streams from the data scratchpad at one word per cycle - 16 activation reads, then per output row one packed-weight read and one result write - and writes each row's dot product back to the scratchpad.

Measured on the SoC (16×16 matrix-vector product, all result vectors compared in-program): software loop 2,489 cycles; direct MMIO mode 605; DMA-fed **86 cycles** including descriptor setup and the status poll. Rows wider than 16 lanes (tiling across multiple accumulate steps) are the designed extension.

Verification harnesses

`src/testbench/` carries per-unit randomized testbenches (ALU: 34,612 vectors; divider: 6,048; MAC: 61 accumulation runs - each against an independent binary reference), the core testbench (bus-only and TCM-fed cores side by side, cross-checked against the simulator's retired count), the full-SoC hello_world test, benchmark harnesses for all three cores, and a generic full-SoC program harness (`tb_vtx1_soc_bench.v`) with phase timestamps via marker stores. `task test-all` runs the suite. == Toolchain and Evaluation

Toolchain

Written in Go under `src/assembler/`:

- **vtxasm** - the assembler: ANTLR grammar, preprocessor (`.INCLUDE`, parameterized `.MACRO`), VLIW packing validation, and output as raw words, 108-bit bundles, Intel HEX, a trit-level objdump disassembly, or `$readmemh` memory images.
- **vtxsim** - the ISA-level simulator with the ratified balanced-ternary semantics; reports dynamic instruction counts, memory traffic and branch behavior.
- **rvsim** + **rvhex** - the RV32IM control: a structural interpreter sharing vtxsim's instrumentation exactly, and an encoder that emits real RV32IM machine code from the same instruction lists, so the binary side of every comparison runs both interpreted and on RTL with asserted results and no external

toolchain.

- The ISA generator emits the Verilog opcode constants and decode tables from `isa.go`.

The evaluation method

The architecture is evaluated in tiers, each with a matched control and machine-checked results (`src/assembler/benchmarks/README.md` holds the full tables and caveats):

Tier A - instruction level. Four kernels, hand-written on both ISAs, asserted by tests: −34% dynamic instructions on three-valued sign arithmetic, −30% on absolute differences, −16% on dot product, −3% on the deliberately neutral control-flow kernel.

Cycle level, issue-width-matched. The same kernels on the VTX1 RTL and on `rv32_triple` (the identical microarchitecture with an RV32IM datapath): parity on the memory-bound kernels, **−10% to −16%** on the three-valued showcases - `ABS` and `CMP` shorten dependent chains that issue width cannot hide. PicoRV32 (vendored unmodified, at its fastest configuration) provides the size-optimized reference point at roughly 3.5× more cycles.

Accelerator level. The ternary-weight MAC array against a matched binary implementation of the same function and against a conventional INT8 multiplier MAC: ternary **weights** halve the array (4,469 vs 9,473 cells) on any substrate; trit **logic** in the 2-bit encoding costs 1.64× its binary equivalent (7,339 vs 4,469) - the quantitative bar for any native ternary substrate.

Tier B - circuit level (`tierb/`, ngspice on SkyWater Sky130 production models). Stock multi-Vt devices provide the three-threshold palette; the ternary inverter family works with an exact 0.9 V middle level; one ternary wire matches the two-wire binary encoding at both speed and energy (1.58 bits/wire at parity, 2 T receivers). The costs, measured: ~1 µA static per stage holding the mid level, and 18 T for voltage-mode negation against 4 T in the 2-bit encoding.

The honest verdict

Balanced ternary buys ~10-16% fewer cycles on sign-heavy dependent code and ~1.58× information density on wires and storage, at a cost of ~1.64× more logic per operation on binary substrates. It is an encoding and interconnect play, not a general compute speedup; the 2-bit-encoded trit datapath VTX1 builds on is cheaper than native voltage-mode ternary CMOS in this process, so the architecture results stand by choice, not compromise. The remaining native-ternary opportunities are interconnect signaling and device research (hard-off at half swing), both recorded in the roadmap.

Known gaps

SQRT and the vector/FP operation sets trap (unimplemented). The cache is not performance-critical in the current memory map and is minimally provisioned. No timing closure has been run - cycle counts are not seconds. The accelerator handles rows up to 16 lanes; tiling is designed but not implemented.